

学号		成绩	
----	--	----	--



沈阳建筑大学

编译原理上机报告

名 称	编译原理实验一
学 院	信息与控制工程学院
专 业	计算机科学与技术
班 级	计算机 1902 班
姓 名	蔡艺洵

2021年 10月 26 日

一、上机目的

通过设计、调试词法分析程序，实现从源程序中分出各种单词的方法；熟悉词法分析程序所用的工具自动机，进一步理解自动机理论。掌握文法转换成自动机的技术及有穷自动机实现的方法。确定词法分析器的输出形式及标识符与关键字的区分方法。加深对课堂教学的理解；提高词法分析方法的实践能力。通过本实验，应达到以下目标：

- 1、掌握从源程序文件中读取有效字符的方法和产生源程序的内部表示文件的方法。
- 2、掌握词法分析的实现方法。
- 3、上机调试编出的词法分析程序。

二、基本原理和上机步骤

词法分析程序的一般设计方案是：

- 1、程序设计语言词法规则 $\Rightarrow$ 正规文法 $\Rightarrow$ FA；
或：词法规则 $\Rightarrow$ 正规表达式 $\Rightarrow$ FA；
- 2、NFA 确定化 $\Rightarrow$ DFA；
- 3、DFA 最简化；
- 4、确定单词符号输出形式；
- 5、化简后的 DFA+单词符号输出形式 $\Rightarrow$ 构造词法分析程序。

要构造词法分析程序，需要提前理解什么是文法、正规表达式和 FA。这是上机的第一步。

一个形式文法 G 是下述元素构成的一个元组 (V_N, V_T, P, S) 。其中 V_N 、 V_T 、 P 、 S 分别是终结符集、非终结符集、开始符、规则产生式。

而确定有限自动机(DFA)理论定义 DFA $M = (Q, \Sigma, t, q_0, F)$ 。其中：

- 1、 Q — 有穷非空状态集；
- 2、 Σ — 有穷输入字母表；
- 3、 t — 映射 $Q \times \Sigma \rightarrow Q$ (单值映射，下态确定)；
- 4、 q_0 — $q_0 \in Q$, 称为开始状态(唯一)；
- 5、 F — 非空终止状态集；

对于 FA，最重要的是给出其映射。可以由状态转换表，状态转换图或者直接给出。

- 1、直接给出： $t(q, a) = q'$ ；
- 2、状态转换表：状态为表列，字母为表行；
- 3、状态转换图：是由一组矢线连接的有限个结点所组成的有向图。每一结点均代表在识别或分

析过程中扫描器所处的状态。它是设计和实现扫描器的一种有效工具，是有限自动机的直观图示。

下面是标识符的状态图举例：

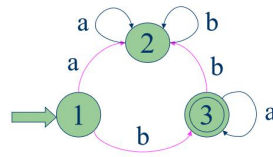


图 1：标识符状态图

(正规式与有限自动机的等价性)定义:对任何两个有限的自动机 $M1$ 和 $M2$,若有 $L(M1)=L(M2)$, 则称 $M1$ 与 $M2$ 等价。

可通过子集法或造表法求解 NFA 的等价 DFA (NFA 的确定化方法)。

正规文法到有穷自动机的转变

其变换规则如下：

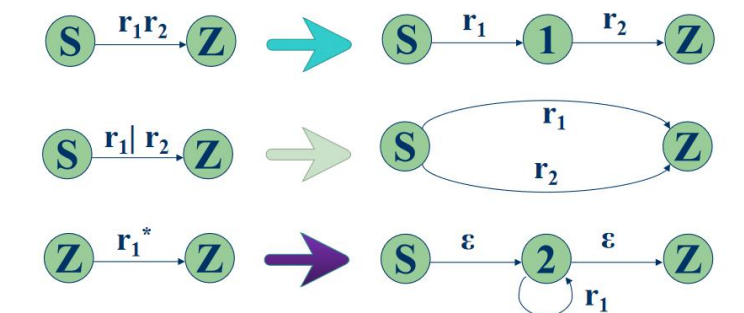


图 2：正规表达式到有穷自动机的变换规则图

三、上机结果

测试数据：

采用 txt 文本来存储相关的测试数据，例如在电脑 D 盘中新建一个名为 try 的 txt 文件，里面存储测试的数据。

输入如下一段：

Begin

a=10;

b=a+20;

end

要求输出如右图。

要求：

(1, Begin)
(24, a)
(21, =)
(25, 10)
(9, ;)
(24, b)
(21, =)
(24, a)
(14, +)
(25, 20)
(9, ;)
(2, end)

识别保留字：Begin、end、if、then、else、while、do；

单词种别码为 1、2、3、4、5、6、7。

符号：,、\、;、{、}、(、)

单词种别码为：8、9、10、11、12、13

运算符包括：+、-、*、/、<=、<>、<、=、>、>=

单词种别码为 14、15、16、17、18, 19, 20, 21, 22, 23。

标识符种别码为 24、整常数种别码为 25.

表 1 测试数据表

单词符号	种别编码	单词值	单词符号	种别编码	单词值
Begin	1	内部字符串二 进制数值表示	+	14	内部字符串二 进制数值表示
end	2		-	15	
if	3		*	16	
then	4		/	17	
else	5		<=	18	
while	6		<>	19	
do	7		<	20	
,	8		=	21	
;	9		>	22	
{	10		>=	23	
}	11		标识符	24	
(	12		整常数	25	
)	13				

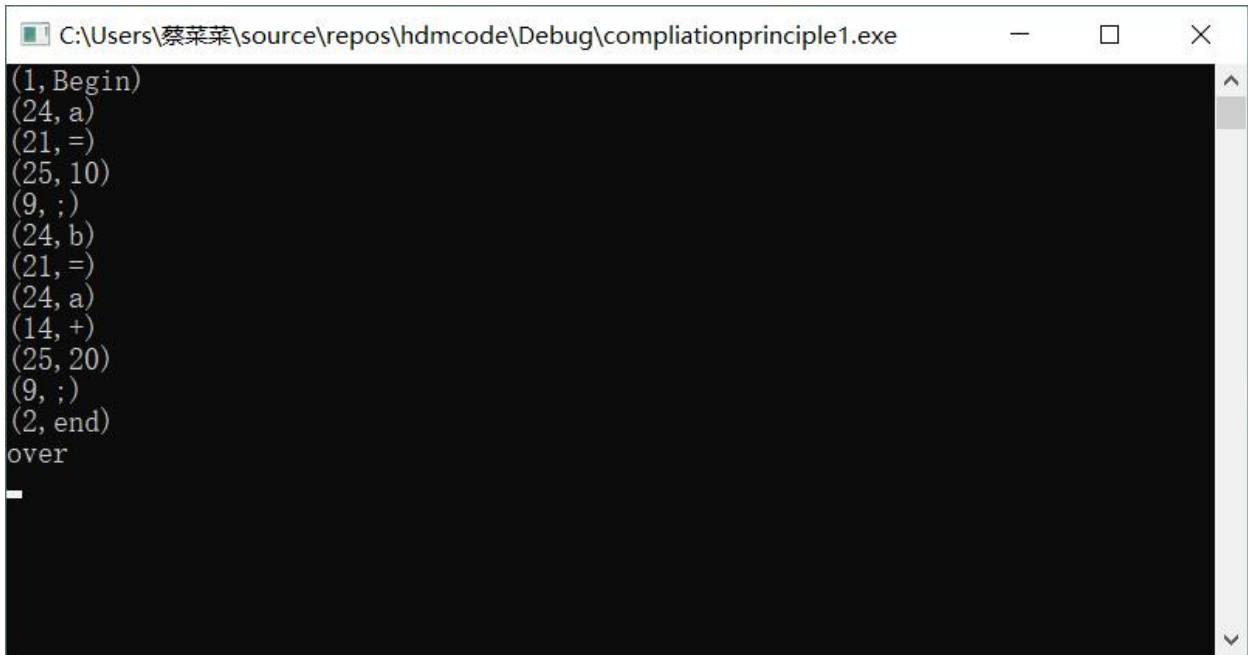
通过 txt 文件输入测试字符集，上述表格是每种字符对应的编码，通过编写 C++程序，自动的对测试字符集进行词法分析。这是词法分析程序的基本思想。词法分析程序所用的全局变量和需调用的函数如下：

1. ch 字符变量，存放当前读进的源程序字符。
2. fgetch()读字符函数，每调用一次从输入文件中读进源程序的下一个字符放在 ch 中，并把读字符指针指向下一个字符。

3. 其他字符编码，不再一一展示，如下图所示：

```
FILE *fp;
char ch;
char *mykey[7] = { (char*)"Begin", (char*)"end", (char*)"if", (char*)"else", (char*)"while", (char*)"do" };
//1 2 3 4 5 6 7
char *border[7] = { (char*)" ", (char*)" ", (char*)" ", (char*)" ", (char*)" ", (char*)" ", (char*)" " };
//8 9 10 11 12 13
char *arithmetic[5] = { (char*)" + ", (char*)" - ", (char*)" * ", (char*)" / " };
//14 15 16 17
char *relation[6] = { (char*)" <=", (char*)" <>", (char*)" <", (char*)" =", (char*)" >", (char*)" >=" };
//18 19 20 21 22 23
char *consts[20];
char *label[20];
int constnum = 0, labelnum = 0;
```

运行结果：



```
C:\Users\蔡某某\source\repos\hdmcode\Debug\compilationprinciple1.exe
(1, Begin)
(24, a)
(21, =)
(25, 10)
(9, ;)
(24, b)
(21, =)
(24, a)
(14, +)
(25, 20)
(9, ;)
(2, end)
over
```

结果和预期的相符，程序运行正确，词法分析成功！

四、讨论与分析

词法分析程序的结果与预期的结果相同，这说明了可以根据不同符号的编码种类，预先画好状态转换图，只要构造出识别语言单词符号的有穷自动机，就很容易构造出识别语言单词符号的词法分析程序。词法分析程序的功能是从左到右扫描源程序字符串，根据语言的词法规则识别出各类单词符号。

对应本次的词法分析程序，代码的业务逻辑如下所示：

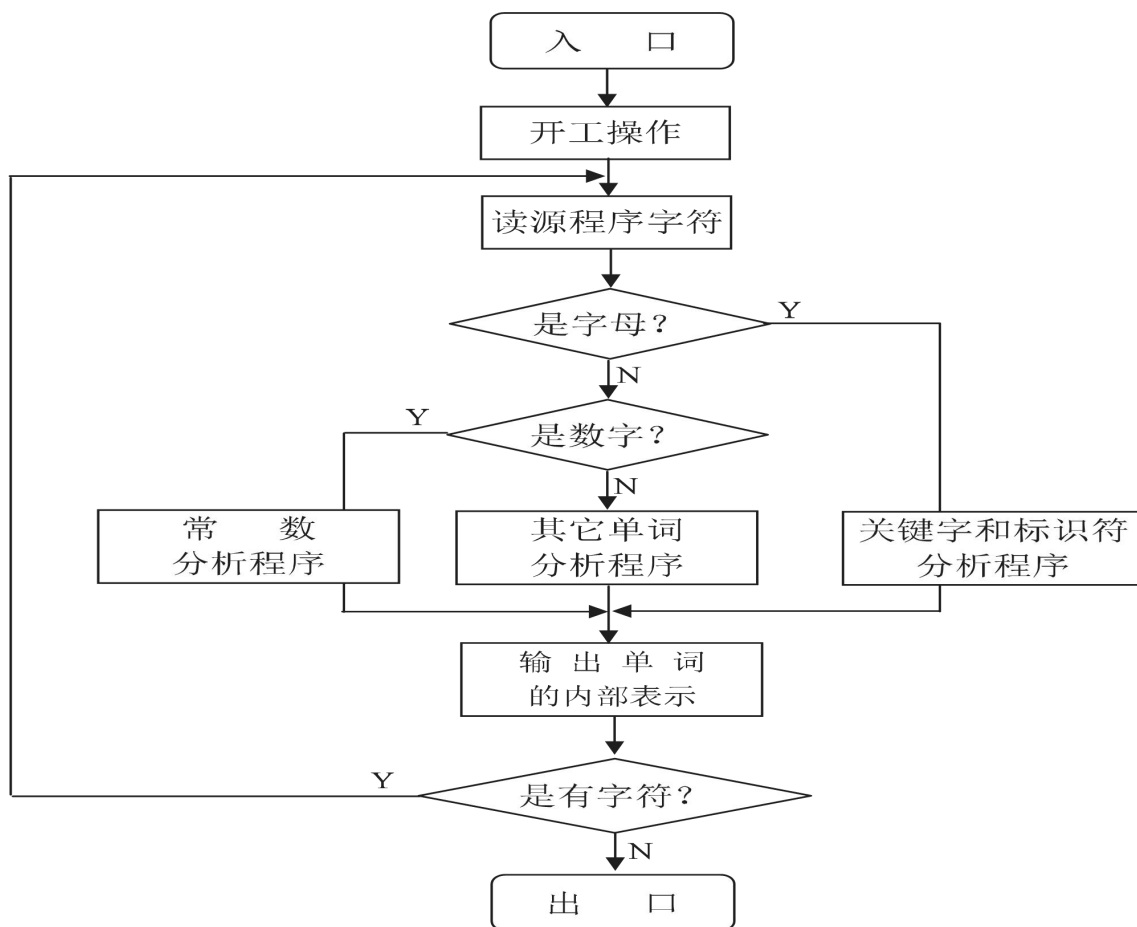


图3 词法分析程序流程图

实验过程记录：出错次数:七八次、出错严重程度：均是细节错误、解决办法摘要：单步调试，找到问题所在的代码位置，监控各个变量的值，与自己思考预期的是否相同，如果不同，思考为何不同，哪个地方可以影响到此变量。

实验总结：上机时间为一下午，三分之一的时间花费在了纸上设计，剩余的时间用在了上机输入和调试，遇到问题就思考哪里出错了。本次上机的整个过程，让我学到了很多，特别是学会了如何构建一个词法分析程序，首先是对相关的定义、理论要了熟于心，要知道单词的两种定义方式（分别是正规式和正规文法）。之后要将其转换成 NFA（非确定有穷自动机），再将其确定化为 DFA（确定有穷自动机），有穷自动机通常表示为状态转换图。最后将 DFA 状态最小化，根据最小化的 DFA 来编写词法分析程序。这就是本次上机的大体流程，教会了我不单单是课本上的理论知识，而是理论知识的实践实现，如何对具体的例子、具体的测试字符，根据要求去划分不同字符的编码种类。去画相关的 NFA、DFA 状态转换图，最后实现词法分析程序。这是理论知识的一次成功实践，对我本人意义重大，也锻炼了我动手编码能力，以及在实现代码的过程中思路的理顺，细节的小心。是一次不可多得的学习经验。

五、源程序清单

```
#include<conio.h>
#include<stdio.h>
#include<ctype.h>
#include<stdlib.h>
#include<string.h>
#define NULL 0

FILE *fp;
char ch;
char *mykey[7] =
{ (char*)"Begin", (char*)"end", (char*)"if", (char*)"else", (char*)"while", (char*)"do" };
//1 2 3 4 5 6 7
char *border[7]= { (char*)"", (char*)";", (char*)"{" , (char*)"}", (char*)"(", (char*)"(") " };
//8 9 10 11 12 13
char *arithmetic[5] = { (char*)"+" , (char*)"-" , (char*)"*" , (char*)"/" };
//14 15 16 17
char *relation[6] = { (char*)"<=" , (char*)"<>" , (char*)"<" , (char*)"=" , (char*)">" , (char*)">=" };
//18 19 20 21 22 23
char *consts[20];
char *label[20];
int constnum = 0, labelnum = 0;

int search(char mysearch[], int wordtype) {
    int i = 0;
    switch (wordtype)
    {
        case 1:
            for (i = 0; i <= 5; i++) {
                if (strcmp(mykey[i], mysearch) == 0) return(i + 1);
            }
            return 0;
        case 2:
            for (i = 0; i <= 5; i++) {
                if (strcmp(border[i], mysearch) == 0) return(i + 1);
            }
            return 0;
        case 3:
            for (i = 0; i <= 3; i++) {
                if (strcmp(arithmetic[i], mysearch) == 0) return (i + 1);
            }
            return 0;
        case 4:
            for (i = 0; i <= 5; i++) {
```

```

        if (strcmp(relation[i], mysearch) == 0) return (i + 1);
    }
    return 0;
case 5:
    for (i = 0; i < constnum; i++) {
        if (strcmp(consts[i], mysearch) == 0) return (i + 1);
    }
    /*consts[i - 1] = (char*)malloc(sizeof(mysearch));
    strcpy(consts[i-1], mysearch);
    constnum++;*/
    return(i);
case 6:
    for (i = 0; i < labelnum; i++) {
        if (strcmp(label[i], mysearch) == 0) return (i + 1);
    }
    /*label[i - 1] = (char*)malloc(sizeof(mysearch));
    strcpy(label[i - 1], mysearch);
    labelnum++;*/
    return(i);
default:
    break;
}
}

```

```

char alph(char buffer) {
    int atype;
    int i = -1;
    char alhatp[20];
    while ((isalpha(buffer)) || (isdigit(buffer)))
    {
        alhatp[++i] = buffer;
        buffer = fgetc(fp);
    }
    alhatp[i + 1] = '\0';
    if (atype = search(alhatp, 1))
        printf("(24,%s)\n", atype, alhatp);
    else {
        atype = search(alhatp, 6);
        printf("(24,%s)\n", alhatp);
    }
    return(buffer);
}

```

```

char digit(char buffer) {

```



```

int i = -1;
char digittp[20];
int dtype;
while ((isdigit(buffer)))
{
    digittp[++i] = buffer;
    buffer = fgetc(fp);
}
digittp[i + 1] = '\0';
dtype = search(digittp, 5);
printf("(25,%s)\n", digittp);
return(buffer);
}

char other(char buffer) {
    int i = -1;
    char othertp[20];
    int otype, otypetp;
    othertp[0] = buffer;
    othertp[1] = '\0';
    if (otype=search(othertp, 3)) {
        printf("(d,%s)\n", 13 + otype, othertp);
        buffer = fgetc(fp);
        goto out;
    }
    if (otype = search(othertp, 4))
    {
        buffer = fgetc(fp);
        othertp[1] = buffer;
        othertp[2] = '\0';
        if (otypetp = search(othertp, 4))
        {
            printf("(d,%s)\n", otypetp, othertp);
            goto out;
        }
        else
            othertp[1] = '\0';
        printf("(d,%s)\n", 17+otype, othertp);
        goto out;
    }
    if (buffer == ':')
    {
        buffer = fgetc(fp);
        if (buffer == '=')

```

```

        printf(":= (2,2)\n");
        buffer = fgetc(fp);
        goto out;
    }
    else
    {
        if (otype = search(othertp, 2))
        {
            printf("(%d,%s)\n", 7+otype, othertp);
            buffer = fgetc(fp);
            goto out;
        }
    }
    if ((buffer != '\n') && (buffer != ' '))
        printf("%c error,not a word\n", buffer);
    buffer = fgetc(fp);
out:    return(buffer);
}

int main(void)
{
    int i;
    for (i = 0; i <= 20; i++) {
        label[i] = NULL;
        consts[i] = NULL;
    }
    if ((fp=fopen("D:\\try.txt", "r"))==NULL) {
        printf("error");}
    else
    {
        ch = fgetc(fp);
        while (ch!=EOF)
        {
            if (isalpha(ch))
                ch = alph(ch);
            else if (isdigit(ch)) {
                ch = digit(ch);}
            else
                ch = other(ch);}
        printf("over\n");
        getchar();
    }
}

```